# Lecture Note: Synchronization Examples in Operating Systems

Prof. Mehdi Pirahandeh

## 1 Introduction

In operating systems, synchronization is crucial for managing concurrent processes that require access to shared resources. Without synchronization, conflicts, data inconsistencies, and unpredictable behavior may arise. This lecture focuses on classical synchronization problems like the Bounded-Buffer, Readers-Writers, and Dining-Philosophers problems, as well as modern synchronization approaches in Linux, POSIX, and alternative programming models.

## 2 Classical Problems of Synchronization

Classical synchronization problems have been used as foundational examples to test and validate synchronization solutions. Each problem illustrates a common concurrency challenge and highlights the necessity of controlling access to shared resources.

### 2.1 Bounded-Buffer Problem (Producer-Consumer)

The Bounded-Buffer Problem, also known as the Producer-Consumer Problem, demonstrates the challenge of managing a finite buffer shared between producer and consumer processes. Key elements are:

- **n Buffers:** Limited slots available for items.
- **Semaphores:** Control access to shared data.
  - **mutex (initialized to 1):** Ensures mutual exclusion.
  - **full (initialized to 0):** Tracks filled buffers.
  - **empty (initialized to n):** Tracks empty buffers.

The producer inserts data into an empty buffer slot, while the consumer retrieves data from a filled buffer. The semaphores `mutex`, `full`, and `empty` synchronize this access, preventing buffer overflow or underflow.

```
Producer:
  wait(empty);
  wait(mutex);
  // Add item to buffer
  signal(mutex);
  signal(full);
```

```

Consumer:
  wait(full);
  wait(mutex);
  // Remove item from buffer
  signal(mutex);
  signal(empty);
```

Listing 1: Producer and Consumer Code in Bounded-Buffer Problem

## 2.2 Readers-Writers Problem

The Readers-Writers Problem highlights a scenario where multiple processes require read and write access to a shared dataset:

- **Readers:** Can read the data simultaneously.
- **Writers:** Require exclusive access to prevent read-write conflicts.

In the first version (First Reader-Writer Problem), readers can starve writers, as continuous reads prevent write access. The second version (Second Reader-Writer Problem) prioritizes writers but can delay readers. This problem is crucial in database management, where read-heavy operations need to be optimized alongside write operations.

```
Semaphore rw_mutex = 1;
Semaphore mutex = 1;
int read_count = 0;

Reader:
  wait(mutex);
  read_count++;
  if (read_count == 1) wait(rw_mutex);
  signal(mutex);
  // Reading
  wait(mutex);
  read_count--;
  if (read_count == 0) signal(rw_mutex);
  signal(mutex);

Writer:
  wait(rw_mutex);
  // Writing
  signal(rw_mutex);
```

Listing 2: Readers-Writers Synchronization Using Semaphores

## 2.3 Dining-Philosophers Problem

The Dining-Philosophers Problem models a situation where multiple processes need access to limited resources, representing five philosophers who either eat or think. Each philosopher needs two forks (or resources) to eat, and they can either be in a **THINKING**, **HUNGRY**, or **EATING** state.

Two types of semaphores manage access:

- **Mutex:** Prevents simultaneous changes to states.

- **Semaphore array:** Controls each philosopher's access to forks.

This problem illustrates deadlock and starvation risks if resources aren't properly managed. To avoid deadlock, philosophers must follow an ordered protocol to acquire resources, like always picking up the lowest-numbered fork first.

# 3 Comparison of Synchronization Problems

| **Problem** | **Processes Involved** | **Synchronization Goal** | **Challenges** |
|---|---|---|---|
| Bounded-Buffer | Producer, Consumer | Coordinate buffer access between producers and consumers | Preventing overflow and underflow |
| Readers-Writers | Readers, Writers | Allow multiple readers or a single writer | Starvation and priority management |
| Dining-Philosophers | Philosophers | Manage resource access (chopsticks) | Deadlock and starvation |

Table 1: Comparison of Classical Synchronization Problems

# 4 Modern Synchronization Techniques

In addition to classical solutions, modern operating systems implement advanced synchronization tools to handle concurrency issues.

## 4.1 Linux Synchronization

Linux provides atomic variables, allowing operations on integers without locking, ideal for low-level tasks needing simple synchronization.

## 4.2 POSIX Synchronization

The POSIX API supports various synchronization primitives, widely used in UNIX and Linux systems.

- **Mutex Locks:** Prevent simultaneous access to critical sections.
- **Semaphores:** Named (shared across processes) and unnamed (within a process) semaphores provide greater flexibility.
- **Condition Variables:** Used with mutexes to wait for specific conditions.

```
pthread_mutex_t lock;
pthread_mutex_init(&lock, NULL);

void critical_section() {
    pthread_mutex_lock(&lock);
    // Critical section
    pthread_mutex_unlock(&lock);
}
```

Listing 3: POSIX Mutex Example for Critical Section

# 5 Alternative Approaches

Alternative synchronization approaches offer additional solutions for concurrency issues.

## 5.1 Transactional Memory

Transactional memory ensures atomic execution of code blocks, eliminating the need for locks. Transactions automatically revert if conditions are not met.

```
atomic {
    count += value;
}
```

Listing 4: Transactional Memory Example

## 5.2 OpenMP

OpenMP enables parallelism with compiler directives. The `#pragma omp critical` directive designates critical sections, ensuring atomic execution.

```
void update(int value) {
    #pragma omp critical
    {
        count += value;
    }
}
```

Listing 5: OpenMP Critical Section Example

## 5.3 Functional Programming

Functional programming avoids mutable state, naturally preventing race conditions. Languages like Erlang and Scala are used in systems where data immutability is essential.

# 6 Example Applications and Use Cases

- **Bounded-Buffer Problem:** Useful in scenarios like file downloading, where multiple consumers retrieve data from a limited buffer.
- **Readers-Writers Problem:** Applied in databases where read-only users need simultaneous access but write access must remain exclusive.

- **Dining-Philosophers Problem:** Illustrates resource allocation challenges, commonly used in deadlock avoidance research.

# 7 Key Term Definitions

To further clarify, the following table summarizes key terms relevant to synchronization.

| **Term** | **Definition** |
|---|---|
| Critical Section | A segment of code where shared resources are accessed, requiring controlled access to prevent conflicts. |
| Race Condition | Occurs when multiple processes access shared data simultaneously, leading to unpredictable outcomes without proper synchronization. |
| Mutex | A locking mechanism to ensure only one process accesses a critical section at a time. |
| Semaphore | A signaling mechanism that controls process access based on resource availability, used for complex synchronization. |
| Deadlock | A situation where processes wait indefinitely for resources held by each other, preventing further progress. |
| Starvation | A situation where a process is perpetually denied access to resources due to other processes continuously being prioritized. |
| Atomic Operation | An indivisible operation that cannot be interrupted, ensuring data consistency without locks. |

Table 2: Important Synchronization Terms

# 8 Conclusion

This lecture covered essential synchronization challenges and solutions, from classical problems like Bounded-Buffer and Dining-Philosophers to modern tools in POSIX, Linux, and functional programming. These concepts and tools provide a foundation for designing reliable, concurrent systems.